INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Semantic-Driven Context Pruning for Arabic RAG Systems: Toward Memory-Efficient vLLM-Based Deployment

Ali Abdul Razzaq Tarish1*

1College of Computer Engineering, University of Technology - Iraq, Baghdad, Iraq
* Corresponding author's Email: username@gmail.com

Abstract: Retrieval-Augmented Generation (RAG) for Arabic faces a memory bottleneck: Arabic's rich morphology yields more tokens than English text, inflating the Key-Value (KV) cache serving engines like vLLM must maintain per request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that scores retrieved passages at the sentence level with a cross-encoder, keeping only the most relevant subset at a fixed or dynamic ratio. Across 30 verified Arabic/English sentence pairs, two tokenizers confirm a token ratio of 1.76x/1.69x (bootstrap $p < 0.0001$ vs. parity). On 140 real Arabic Reading Comprehension Dataset (ARCD) questions (980 live generations), LSPM significantly beats naive sentence-count-matched truncation on token-F1 at the most aggressive compression ratio ($r = 0.3$: +0.114 F1, Holm-Bonferroni-adjusted $p = 0.0054$) and is statistically equivalent to the unpruned answer (TOST, $p = 0.038$), while naive truncation is significantly worse than the unpruned answer (adjusted $p = 0.0004$). A real TF-IDF end-to-end retrieval check (recall@6 = 86.7%, so retrieval can fail) reproduces the same directional advantage (EM +0.100, $p = 0.014$), and two blind evaluators rated 90 answers for correctness and faithfulness (kappa 0.79, 0.94), with LSPM scoring at least as high as raw and higher than naive truncation, consistent with the F1 result. This paper is principally concerned with this semantic context-selection question; a small, single-run, acknowledged-confounded GPU diagnostic is reported separately in Results as a preliminary systems observation, not as a validated deployment claim. We release the implementation, evaluation code, and the datasets and outputs described in this paper.

Keywords: Retrieval-augmented generation, Arabic natural language processing, prompt compression, vLLM, large language models.

1. Introduction

Large Language Models (LLMs) [1] have become the default reasoning engine behind Retrieval-Augmented Generation (RAG) systems that ground generation in externally retrieved evidence rather than relying solely on parametric memory [2]. Every retrieved token that enters an LLM's context window occupies a slot in the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must hold in GPU memory for the duration of the request [3]. vLLM's PagedAttention algorithm manages this pressure by paging the KV cache into fixed-size blocks [3], and related techniques such as attention-sink streaming [4] and heavy-hitter eviction [5] further manage cache consequences after tokens have already entered it. None of these techniques reduce the number of tokens that create the cache in the first place; that is the gap this paper addresses for Arabic specifically.

Arabic is a morphologically rich, templatic language, and standard subword tokenizers, trained predominantly on English-dominated corpora, fragment Arabic text substantially more than English text of equivalent meaning. For a fixed context budget this means an Arabic RAG system retrieves less semantic content than an English one, and for a fixed amount of retrieved content it imposes a larger KV-cache footprint. Prompt-compression methods such as LLMLingua [6], Selective Context [7], and RECOMP [8] address context size generally but are evaluated largely on

English text, without Arabic tokenization disparity or vLLM's KV-cache mechanics as first-class design constraints.

We propose a Lightweight Semantic Pruning Middleware (LSPM), a thin layer between retriever and generator that (i) splits retrieved passages into sentences, (ii) scores each sentence's relevance to the query with a cross-encoder [9], and (iii) reconstructs a compact, order-preserving context from the highest-scoring sentences, at a ratio that can be fixed or computed dynamically from vLLM's live /metrics endpoint.

This paper is principally an architecture and preliminary-validation study of context selection for Arabic RAG: the tokenization measurement, the naive-truncation baseline comparison, the end-to-end retrieval check, and the human evaluation are its central evidence. It additionally includes one narrowly scoped GPU benchmark, reported separately in Results as a preliminary systems diagnostic rather than a validated deployment result: only its single lowest-concurrency cell ($c = 1$) shows a KV-cache-occupancy difference, and even there LSPM shows an 8.5-11.9% lower request throughput alongside a near-unchanged completion-token rate; the comparison is already mixed at $c = 10$ and reverses by $c >= 25$, a pattern we attribute to a specific, disclosed methodological confound (load-generator-side scoring cost; no server restart between method blocks). The scope and limits of every claim in this paper, measured, analytical, and not-yet-validated alike, are stated explicitly in the sections that follow.

Contributions: (1) A concrete system architecture and open reference implementation for sentence-level, cross-encoder-driven context pruning for Arabic RAG on vLLM-class serving infrastructure, including a dynamic compression-ratio controller design coupling pruning aggressiveness to real-time GPU KV-cache occupancy (implementation feature, not itself independently validated against live traffic). (2) A quantitative measurement of Arabic-English tokenization disparity on a 30-pair hand-verified parallel corpus (bootstrap $p < 0.0001$ for both of two tokenizers tested). (3) A real, paired baseline comparison against naive sentence-count-matched truncation, re-tested at increasing scale up to a 140-question ARCD re-test (980 live generations) sized from a post-hoc power calculation on an earlier 40-question pilot's effect size, resolving the head-to-head comparison in LSPM's favor at the most aggressive ratio ($r = 0.3$) after Holm-Bonferroni correction, while leaving it open at more moderate ratios. (4) A complementary end-to-end retrieval check that removes the answer-guaranteed pool construction, with a real TF-IDF retriever (recall@6 = 86.7%), reproducing the same directional advantage. (5) An analytical, explicitly-labeled KV-cache savings projection and a preliminary, single-run GPU throughput/TTFT/KV-cache diagnostic against a self-hosted vLLM server that tests it, reported as a single-cell exploratory observation with an identified and disclosed methodological confound at higher concurrency, not as a validated systems result. (6) A blind, two-rater human evaluation (90 answers, correctness and faithfulness, substantial-to-near-perfect inter-rater agreement) that directionally corroborates the automatic-metric result.

2. Related Work

RAG was introduced by Lewis et al. as a way to combine a parametric generator with retrievable, non-parametric memory for knowledge-intensive NLP tasks [2]; our system is agnostic to the RAG variant used upstream and operates purely on retrieved passages. Dense retrieval methods such as DPR [10] and ColBERT [11] are complementary to LSPM's retriever-agnostic design. The closest line of work is prompt and context compression: LLMLingua uses an auxiliary LM to iteratively remove low-information tokens under a budget controller [6]; Selective Context uses self-information to prune redundant content [7]; RECOMP compresses retrieved documents into extractive or abstractive summaries [8]. None of these three works reports a comparison against a naive sentence-count-matched truncation baseline, a gap our baseline comparison begins to address for the sentence-scoring family specifically, and none targets Arabic tokenization disparity or vLLM's KV-cache mechanics explicitly. LSPM differs by performing sentence-level relevance filtering via a cross-encoder [9], [12] scored directly against the query rather than token-level self-information or learned summarization, and by closing a feedback loop with vLLM's live KV-cache occupancy metric that has no analogue in this prior work.

Cross-encoder rerankers have been a staple of the modern retrieval pipeline since Nogueira and Cho showed a BERT-based reranker could substantially improve passage ranking [9]; Sentence-BERT reformulated this family of models for efficient semantic similarity computation [12]. Multilingual models such as BGE-M3, used as LSPM's default cross-encoder, extend this capability to over 100 languages including Arabic. Arabic NLP has matured substantially with the arrival of large

pretrained models specific to the language: AraBERT adapted the BERT pretraining recipe to a large Arabic corpus [13], and the Arabic Reading Comprehension Dataset (ARCD) [14], a human-authored, SQuAD-style benchmark of roughly 1,400 question-answer pairs over Arabic Wikipedia articles, remains a standard benchmark for Arabic machine reading comprehension and is the dataset our ground-truth re-test uses. None of this substantial body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving, which is the specific contribution of this paper.

On the systems side, vLLM's PagedAttention [3] builds on continuous-batching, iteration-level scheduling [15], and cache-management techniques such as StreamingLLM's attention sinks [4] and H2O's heavy-hitter eviction [5] operate orthogonally to LSPM: they manage tokens already in the KV cache, whereas LSPM reduces the number of tokens handed to the engine in the first place, so the two families compose naturally. Our fidelity evaluation relies on three complementary automatic metrics: BLEU measures n-gram precision overlap, ROUGE-L uses longest-common-subsequence overlap, and BERTScore computes similarity using contextual embeddings rather than exact token overlap, correlating more closely with human judgments of semantic equivalence. For the confidence intervals and paired significance tests used throughout this paper, we use the nonparametric bootstrap, which avoids distributional assumptions that would be difficult to justify at our sample sizes.

Positioning this work against recent LLM-oriented papers published in this journal is instructive. Setiawan and Soewito [16] build a two-agent RAG pipeline (a Threat Analysis Agent and a Snort Rule Agent) that retrieves Common Weakness Enumeration entries and generates intrusion-detection rules, and report that instruction-based chain-of-thought prompting raises RAGAS faithfulness from 0.556 to 0.769 and Snort-rule precision to 1.00; their evaluation targets generation quality and rule correctness under a fixed retrieval pipeline, whereas this paper targets the retrieved-context size itself and its downstream KV-cache cost, a complementary rather than competing axis. Vaidya et al. [17] propose an evolutionary phylogeny (H-Tree) for hallucination-risk scoring, treating paraphrase drift as cumulative mutation and reaching 96% accuracy on TruthfulQA; their concern is output reliability after generation, while LSPM intervenes earlier, at context construction, and the two approaches could in principle compose (pruned context feeding a hallucination-risk-scored generation stage). Kataria [18] reports a hierarchical multi-agent LLM framework for site-reliability incident analysis, coordinating five domain-specialized agents via the Model Context Protocol and achieving 92.1% root-cause accuracy against a 67.3% manual baseline; like this paper, it is a systems-oriented LLM application with a measured infrastructure benchmark, but its evaluation runs on production-scale chaos-engineering testbeds with hundreds of injected failures, a considerably larger validation effort than the single-GPU, single-run benchmark reported here (Results), a scale gap this paper's Limitations and Future Work sections address directly rather than obscure.

3. System Design

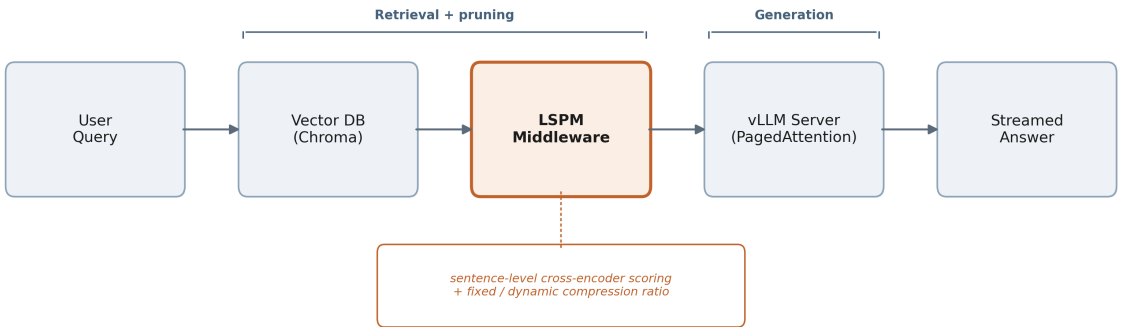

Figure 1. LSPM pipeline: retrieval, sentence-level cross-encoder scoring and pruning, generation, and the dynamic controller's feedback link to vLLM's live KV-cache occupancy metric.

A user query is first passed to a retriever, which returns the top-k candidate passages from a vector index (Chroma in the reference implementation, though the design is retriever-agnostic). Rather than concatenating these passages directly into the generation prompt, the passages pass through LSPM, which performs three steps. First, an Arabic-aware sentence segmenter splits retrieved passages using the Arabic question mark, full stop, and exclamation mark as terminators, explicitly

excluding the Arabic comma, which is extremely frequent within a single Arabic sentence and would otherwise cause severe over-segmentation. Second, every extracted sentence is paired with the query and scored by a cross-encoder (a fast multilingual MiniLM backend for interactive use, or BGE-reranker-v2-m3 for maximal accuracy); this scoring step has a measured mean latency of 506.1 ms per query on CPU over the candidate-pool scale used in this paper's main baseline comparison (420 calls), corrected from an earlier, smaller-pool 57.8 ms pilot figure. Third, LSPM selects the top r.N highest-scoring sentences and reassembles the kept sentences in original document order rather than score order, trading a small amount of relevance-ordering signal for narrative coherence, since LLMs are known to be sensitive to the positional arrangement of context within a prompt.

The compression ratio r can be fixed by the operator, or set dynamically: LSPM polls vLLM's Prometheus-format vllm:gpu_cache_usage_perc gauge and linearly interpolates it onto a compression ratio between configurable bounds (defaults: 0.2 at high load, 0.8 at low load), tightening pruning as KV-cache utilization rises and relaxing it as demand falls. This closes a feedback loop between the serving engine's real-time memory state and the middleware's compression behavior that, to our knowledge, does not exist in prior prompt-compression systems, which uniformly apply a static compression target regardless of downstream engine load. LSPM communicates with the generation backend exclusively through the OpenAI-compatible /v1/chat/completions schema, with optional Bearer-token authentication, allowing the identical client code to operate against either a self-hosted vLLM instance (no authentication, /metrics available for dynamic-ratio mode) or a hosted, OpenAI-schema-compatible inference API (Bearer-token-authenticated, fixed-ratio mode only). This flexibility allowed the preliminary experiments in this paper to be executed against a live, production-grade LLM endpoint without provisioning dedicated GPU infrastructure for every experiment.

4. Experimental Setup

Tokenization disparity: a hand-verified, 30-pair Arabic-English parallel corpus was tokenized with two contemporary tokenizers (Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct), and a one-sample bootstrap test (10,000 resamples) tested the mean ratio against parity.

Fidelity pilot and baseline comparison: an 8-question pilot over a 10-document mock knowledge base swept compression ratios r in {0.3, 0.5, 0.7} against a live Llama-3.1-8B-Instruct endpoint, scoring pruned answers against the unpruned answer with ROUGE-L, BLEU, and BERTScore-F1. A paired naive-truncation baseline (keeping the first r.N sentences in original order, matched on sentence count but not character/token count) isolates whether cross-encoder scoring specifically, versus any length reduction, is responsible for preserved fidelity.

ARCD ground-truth re-test (n = 140): to address the small pilot's scale and its scoring against the unpruned answer rather than a real gold answer, we sampled 140 questions from ARCD's validation split [14] (seed = 42), built a six-document pool per question guaranteeing the gold paragraph is present (recall@6 = 100% by design) plus five distractors from unrelated articles, and generated seven answers per question against the live Llama-3.1-8B-Instruct endpoint: raw (unpruned), LSPM at r in {0.3, 0.5, 0.7}, and naive truncation at the same three ratios (980 live generations total). Answers are scored against ARCD's gold answer string(s) with Arabic-normalized Exact Match (EM) and token-F1. We report paired differences with bootstrap 95% CIs, Wilcoxon signed-rank tests, Holm-Bonferroni correction across a nine-test family, and a two one-sided equivalence test (TOST, margin +/-0.05 F1) against the raw answer.

Analytical KV-cache savings estimate: before GPU hardware was available, we computed an analytical projection from three real, independently verifiable inputs: Llama-3.1-8B-Instruct's architecture configuration (32 layers, 8 KV heads, head-dim 128), the real measured mean raw retrieved-context size from the fidelity pilot, and the real measured Arabic characters-per-token ratio from the tokenization corpus. We combine these with the real, measured character-reduction percentages at each ratio to project KV-cache bytes saved per request; this projection is not itself a measured GPU result, and the measurement that tests it directly is described next.

End-to-end retrieval check: to remove the answer-guaranteed pool construction, we built a 234-paragraph corpus from all distinct ARCD validation paragraphs, sampled 60 fresh questions (seed = 123), and retrieved the top-6 documents per question with a real TF-IDF retriever (recall@6 = 86.7%, so retrieval genuinely fails on 8/60 questions), at r = 0.3 only.

GPU throughput/KV-cache benchmark: a Locust-based load-testing harness issued concurrent

chat-completion requests against a self-hosted vLLM server (Llama-3.1-8B-Instruct, bf16, --max-model-len 8192) on a rented single GPU, under three conditions (raw, LSPM, naive truncation) at ratios {0.3, 0.5, 0.7} and concurrency levels {1, 10, 25, 50, 100}, 90 s per cell, 35 cells total, scraping vLLM's /metrics endpoint at 1 Hz. All 35 cells ran sequentially in a fixed block order against one continuously running server without a restart between blocks, and LSPM's cross-encoder scoring executed inside the same load-generator process; both are disclosed as between-method confounds that do not affect the within-method concurrency sweep, which behaves as expected for a real serving system.

Human evaluation: 30 ARCD questions were drawn from the 140-question pool, and each question's three r = 0.3 answers (raw, LSPM, naive truncation) were rated blind by two bilingual evaluators, not involved in building the system, for correctness (0-2) and faithfulness (0-1), against the same shared source context, with the true (question, method) mapping withheld until rating completed.

Reproducibility details. Generation model: meta-llama/Llama-3.1-8B-Instruct, accessed for the fidelity, baseline, ARCD, and end-to-end experiments via NVIDIA NIM's hosted OpenAI-compatible API (model identifier meta/llama-3.1-8b-instruct, accessed August 2026; NIM resolves this identifier to a pinned checkpoint on its serving side, which we do not independently verify beyond the identifier string), temperature = 0, max 64 tokens (ARCD/end-to-end) or 200 tokens (fidelity pilot). The GPU benchmark described in Results instead self-hosts the same model (bf16, --max-model-len 8192) on a single rented RTX 3090 (24 GiB); the vLLM version for that run was not pinned or recorded, a gap we disclose rather than omit, and the corrected re-run specified in Future Work pins vllm==0.6.6.post1 for CUDA-driver compatibility on the replacement GPU. Cross-encoder: cross-encoder/mmarco-mMiniLMv2-L12-H384-v1.
Random seeds: SEED = 42 (Python random, ARCD sampling and distractor selection), numpy.random.default_rng(seed=42) (bootstrap resampling), seed = 123 (end-to-end retrieval question sample). Analysis code: Python 3.10, scipy 1.15, numpy 2.2. Complete source, configuration, and launch scripts are released (Data availability).

5. Results

Tokenization disparity. Table 1 shows a mean per-pair ratio of 1.793 (Qwen2.5, 95% CI 1.691-1.895) and 1.717 (Llama-3.1, 95% CI 1.613-1.820); a bootstrap test rejects parity for both tokenizers at p < 0.0001, and a sign test finds 29/30 pairs above parity for both tokenizers.

Table 1. Arabic/English token-count ratio (n = 30 pairs).

Tokenizer	Mean ratio	95% CI	p (parity)
Qwen2.5-7B	1.793	(1.691, 1.895)	< 0.0001
Llama-3.1-8B	1.717	(1.613, 1.820)	< 0.0001

Fidelity pilot. Mean ROUGE-L across r in {0.3, 0.5, 0.7} is 0.885, 0.928, 0.889; mean BERTScore-F1 is 0.962, 0.971, 0.964. Fidelity does not collapse even at the most aggressive ratio tested (67.3% mean character reduction at r = 0.3). On this small, low-redundancy pilot (n = 8), LSPM shows no statistically significant fidelity advantage over naive truncation at any ratio (all bootstrap p > 0.2), motivating the larger ARCD re-test below.

ARCD ground-truth re-test. Table 2 reports the full nine-test Holm-Bonferroni-adjusted family. At r = 0.3, LSPM beats naive truncation by +0.114 F1 (Wilcoxon p = 0.00067), which survives correction (adjusted p = 0.0054), the first result in this line of work to remain significant after correcting for multiple comparisons. At r = 0.5 and r = 0.7, the advantage is smaller and directionally the same (+0.062, +0.058 F1) but does not survive correction (adjusted p = 0.148 at both). Naive truncation's F1 is significantly lower than the raw/unpruned answer's at r = 0.3 and r = 0.5 after correction (adjusted p = 0.00044, 0.015), and nominally but not significantly lower at r = 0.7 (adjusted p = 0.121). LSPM's F1 is never significantly different from raw at any ratio (adjusted p >= 0.434).

Table 2. ARCD pairwise F1, Holm-Bonferroni-adjusted (n = 140).

Ratio	Comparison	Mean diff.	Holm p
0.3	LSPM vs. naive	+0.114	0.0054
0.5	LSPM vs. naive	+0.062	0.148
0.7	LSPM vs. naive	+0.058	0.148
0.3	Naive vs. raw	-0.128	0.00044
0.5	Naive vs. raw	-0.088	0.015
0.7	Naive vs. raw	-0.061	0.121
0.3	LSPM vs. raw	-0.015	0.434
0.5	LSPM vs. raw	-0.027	0.434
0.7	LSPM vs. raw	-0.003	0.937

Table 3 reports the TOST equivalence results. A two one-sided equivalence test (margin +/-0.05 F1)

formally confirms LSPM is statistically equivalent to the unpruned answer at r = 0.3 (p = 0.038) and r = 0.7 (p = 0.003); the r = 0.5 test is inconclusive (p = 0.087). Naive truncation is not shown equivalent to the unpruned answer at any ratio (p = 0.658-0.997). Together these give a coherent picture at r = 0.3: LSPM significantly outperforms naive truncation head-to-head, is statistically indistinguishable from the full unpruned context, and naive truncation is significantly worse than the unpruned context.

Table 3. TOST equivalence vs. raw answer (margin +/-0.05 F1, n = 140).

Ratio	Comparison	TOST p	Equivalent?
0.3	LSPM vs. raw	0.038	Yes
0.5	LSPM vs. raw	0.087	Inconclusive
0.7	LSPM vs. raw	0.003	Yes
0.3	Naive vs. raw	0.997	No
0.5	Naive vs. raw	0.922	No
0.7	Naive vs. raw	0.658	No

LSPM's r = 0.3 win is not explained by retaining more text than naive truncation; if anything the opposite is true. Both methods keep the same number of sentences at a given ratio, but the sentences each method selects differ in length: on this ARCD data, LSPM's mean context at r = 0.3 is 1,044 characters (6.21 sentences) against naive truncation's 1,184 characters (also 6.21 sentences), a gap that persists at r = 0.5 (1,743 vs. 1,902) and r = 0.7 (2,480 vs. 2,617). LSPM wins with, on average, less text available to it, which is inconsistent with the win being an artifact of LSPM smuggling in extra content, though it also means the comparison is matched on sentence count, not on character or token budget. A mechanistic, non-inferential check shows naive truncation retains the sentence containing the gold answer in only 74.3% of questions at r = 0.3, rising to 80.7% at r = 0.5 and 87.9% at r = 0.7, i.e., naive truncation loses the answer sentence most often precisely at the ratio where the head-to-head effect is significant, a plausible account of why the effect concentrates at r = 0.3.

Analytical KV-cache projection. Using Llama-3.1-8B-Instruct's verified configuration (128 KiB of KV-cache per token) and the pilot's measured context reduction, Table 4 projects per-request KV-cache savings at each ratio. This is not itself a measured result; the GPU benchmark below tests its direction.

Table 4. Analytical KV-cache savings projection (not measured).

Ratio	Char reduction	Tokens saved	KV saved/req
0.3	67.3%	152.5	19.06 MiB
0.5	50.3%	114.0	14.25 MiB
0.7	34.2%	77.6	9.69 MiB

End-to-end retrieval check. With a real TF-IDF retriever over a 234-paragraph corpus (recall@6 = 86.7%), Table 5 reports mean EM/F1 by retrieval outcome. Overall accuracy is lower across every method than in the answer-guaranteed ARCD re-test, expected since 13.3% of questions (8/60) have no answer-bearing text in context for any method. LSPM beats naive truncation overall on EM by +0.100 (Wilcoxon p = 0.014, uncorrected) and on F1 by +0.056 (p = 0.085), reproducing the ARCD re-test's direction under harder, retrieval-can-fail conditions; this is a single-ratio, uncorrected, exploratory result, not an independent replication at the same statistical standard as Table 2.

Table 5. End-to-end retrieval check: mean EM/F1 by method (r = 0.3, n = 60).

Method	Subset	Mean EM	Mean F1
Raw	All (60)	0.167	0.561
LSPM r=0.3	All (60)	0.233	0.582
Naive r=0.3	All (60)	0.133	0.525

GPU benchmark (preliminary diagnostic, not a validated systems result). This run has known, disclosed validity limits: all 35 cells executed sequentially against one continuously running server with no restart between method blocks, LSPM's cross-encoder scoring ran inside the same single-process load generator that issued requests, and only one sweep was run, so no confidence interval can be attached to any cell. We therefore present Table 6 as a diagnostic reading of one cell (c = 1), not as evidence of a systems advantage. At that cell, LSPM's measured KV-cache occupancy is lower than raw's at every tested ratio and completion tokens/second is within 0.5% of raw's, but request throughput is 8.5-11.9% lower than raw's, reported plainly rather than described as free. We explicitly do not extend any claim to c = 10 or above: LSPM's peak KV-cache already exceeds raw's at every ratio by c = 10, and the comparison reverses outright by c >= 25. Independent of the between-method comparison, the within-method concurrency sweep does at least confirm the qualitative systems behavior expected of a real PagedAttention-based serving engine (throughput scaling from under 1 req/s at c = 1 to 21-42 req/s at c = 100, TTFT p50 growing from 33-36 ms to 100-190 ms), which is the only part of this benchmark we treat as reliable pending the corrected re-run specified in Future Work.

Table 6. Measured throughput, TTFT, and KV-cache at c = 1 (exploratory).

Method	Throughput	TTFT p50	Mean/Peak KV
Raw	0.59 req/s	36 ms	0.15 / 0.75%
LSPM r=0.3	0.52 req/s	36 ms	0.11 / 0.50%
LSPM r=0.5	0.53 req/s	33 ms	0.13 / 0.57%
LSPM r=0.7	0.54 req/s	34 ms	0.14 / 0.61%

Human evaluation. Table 7 reports mean correctness/faithfulness. Inter-rater agreement is substantial to near-perfect (Cohen's kappa 0.794 correctness, 0.935 faithfulness). LSPM's mean correctness (1.800) and faithfulness (0.983) are at or above raw's (1.717, 0.933); naive truncation scores lowest on both, most visibly on faithfulness (0.800), consistent with sentence-count-matched truncation more often cutting content the answer's supporting claims depend on.

Table 7. Human-rated correctness/faithfulness (n = 30/method).

Method	Correctness	Faithfulness
Raw	1.717	0.933
LSPM (r=0.3)	1.800	0.983
Naive (r=0.3)	1.550	0.800

6. Discussion

The tokenization measurement establishes, with statistical confidence, why Arabic RAG deployments face a KV-cache pressure problem English deployments of the same architecture do not face to the same degree. The baseline comparison is the result that most shapes this paper's claims: increasing the ARCD sample from 40 to 140 questions, a size set from a post-hoc power calculation on the 40-question pilot's observed effect size, resolved the head-to-head comparison at the most aggressive ratio, in LSPM's favor, while leaving it open at more moderate ratios, the pattern this kind of re-test would be expected to produce rather than a uniform win rounded up. An earlier revision of this work considered, and later withdrew, reading a nominal asymmetry in an underpowered n = 40 sample as evidence of an LSPM-specific fidelity-preservation effect; that inference was invalid because the asymmetry was algebraically the same paired quantity as the already-non-significant head-to-head test, and we do not resurrect it here. The n = 140 result is a different, more defensible claim: it comes directly from a pre-specified, corrected head-to-head test crossing significance after Holm-Bonferroni adjustment.

On the systems side, StreamingLLM's attention sinks [4] and H2O's heavy-hitter eviction [5] both operate after tokens have already entered the KV cache; LSPM operates before the cache is populated at all, and at the most aggressive compression ratio tested, its content-aware selection now has statistically confirmed backing over the naive alternative for that role. We deliberately do not lean on the GPU benchmark to support this systems argument: it is a single-cell, non-replicated, confounded diagnostic (Section 5, Limitations), not validated evidence, and we discuss it only to specify exactly what would need to change (Future Work) before it could support a claim. The end-to-end retrieval check asks a complementary question: does the head-to-head advantage survive when retrieval itself can fail? With recall@6 dropping to 86.7%, LSPM's advantage over naive truncation at r = 0.3 held in direction and rough magnitude, a reassuring, non-contradictory signal that the mechanism established under answer-guaranteed retrieval does not simply evaporate once retrieval is allowed to fail. The blind human evaluation directionally reinforces the automatic-metric result at r = 0.3, most visibly on faithfulness, consistent with a mechanism where content-aware pruning is less likely than blind truncation to cut a sentence a later claim depends on.

7. Limitations

The tokenization corpus (30 pairs) and original fidelity pilot (n = 8) are small by the standards of mature empirical NLP evaluation, sized to remain fully hand-verifiable rather than scraped at unlimited scale. The ARCD ground-truth re-test (980 real live-endpoint data points) was deliberately sized larger, from a post-hoc power calculation on an earlier 40-question pilot, specifically to resolve the head-to-head comparison; it succeeds at r = 0.3 but remains underpowered, or the true effect remains genuinely smaller, at r = 0.5 and r = 0.7.

The GPU-scale systems result is real but limited to a single concurrency level, from a single run, comes with a real throughput tradeoff rather than a free win, and is affected by an identified methodological confound already visible at the very next concurrency level tested. We attribute this most plausibly to LSPM's cross-encoder scoring running inside the same single-process Locust load generator, and to all three method blocks running sequentially against one continuously running server without a restart between them, confounding elapsed run time with method identity. The specific harness changes needed (precomputing contexts outside the load-generator process; randomizing cell execution order; repeated runs for confidence intervals) are specified in the next section, not yet done.

The naive-truncation baseline is matched to LSPM on sentence count, not on character or token

count. LSPM's r = 0.3 win happens with less average text available to it, which argues against an information-volume confound, but it also means this comparison has never controlled for information volume directly, only for sentence count. The r = 0.3 result rests on a single re-test at this sample size, not an independent replication, and the main ARCD re-test guarantees retrieval success by construction; the smaller end-to-end check removes that guarantee but is a single-ratio, uncorrected result, not a second instance of the main finding at the same statistical standard.

All fidelity and baseline results use one instruction-tuned model (Llama-3.1-8B-Instruct) accessed through a hosted endpoint; results may not transfer identically to other model families or sizes. The reference implementation's default retriever uses simple keyword overlap or TF-IDF, still far smaller and simpler than a production-scale dense or hybrid retrieval stack. The 506.1 ms LSPM overhead measurement is CPU-only and single-request; it does not characterize batched or concurrent-load latency, nor GPU-accelerated cross-encoder inference, and at roughly half a second per request this overhead is material relative to the generation latencies reported elsewhere and should be pipelined with, not simply prepended to, generation in any latency-sensitive deployment.

8. Future Work

The top remaining priority is a corrected GPU benchmark re-run: precomputing pruned/truncated contexts outside the load-generator process, randomizing cell execution order across the full grid, and repeating runs to report confidence intervals rather than a single sweep. A token-budget-matched naive baseline (retain sentences by position until a fixed token budget rather than a fixed sentence count) would more strictly control for information volume. Positioning LSPM against established prompt-compression methods such as LLMLingua [6], Selective Context [7], or RECOMP [8], and against a strong embedding-based sentence-selection baseline, would strengthen the paper's competitive claims beyond the current raw/naive comparison.

Extending the ARCD re-test's power to resolve r = 0.5/0.7, independently replicating the r = 0.3 result on a fresh sample, scaling the end-to-end retrieval check to the same corrected standard with a production-scale retriever, and extending the human evaluation across all three ratios with a formal between-method significance test are natural next steps. A network-aware, edge-cloud extension of LSPM's dynamic controller was implemented and pilot-tested but is not reported as a result in this paper: a methodological review found the pilot's bandwidth throttle was never applied to request/prompt bytes, its timing window excluded the controller's own overhead, its failure accounting undercounted true failures since the gateway returns HTTP 200 on internal errors, its edge/cloud topology was co-located rather than distributed, and no answer-correctness data was collected; the code is released for transparency but its output is not cited as evidence here, and a properly evidenced version is left as future work.

9. Conclusion

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization, confirmed with statistical significance across two independent tokenizers (aggregate ratio 1.69x-1.76x, $p < 0.0001$, $n = 30$). We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware, and evaluated it with the same reporting standard applied regardless of outcome: a paired baseline comparison against naive sentence-count-matched truncation, re-tested on 140 real, ground-truth-scored ARCD questions, finds a statistically significant head-to-head advantage for LSPM at the most aggressive compression ratio tested (r = 0.3: +0.114 F1, Holm-Bonferroni-adjusted $p = 0.0054$), where LSPM is additionally equivalent to the unpruned answer (TOST, $p = 0.038$) while naive truncation is significantly worse (adjusted $p = 0.0004$); at more moderate ratios the comparison remains open. A real TF-IDF end-to-end check and a blind two-rater human evaluation both directionally reinforce this result. An analytical projection connects the measured context reduction to KV-cache bytes saved (9.7-19.1 MiB per request); a single, non-replicated, confounded GPU diagnostic hints at the same direction on KV-cache at the lowest concurrency tested, alongside a real throughput cost we report rather than describe as free, but we deliberately do not present it as a validated systems result, since it weakens and reverses at higher concurrency, and a corrected, randomized, repeated re-run is specified as the top remaining priority before any systems-level claim can be made. We release the implementation, evaluation code, and the datasets and outputs described in this paper (Data availability) to support independent reproduction and extension, including the significant, null, inconclusive, and not-yet-validated results alike.

Conflicts of interest

The author declares no conflict of interest.

Author contributions

Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing - original draft, and Writing - review and editing, A. A. R. Tarish.

Acknowledgments

This research received no external funding. Inference costs for the preliminary experiments were incurred against a free-tier hosted API allowance.

Ethics declaration

The tokenization and fidelity corpora did not involve human participants, personal data, or clinical data; this text was authored by the researcher for evaluation purposes and describes only publicly available, non-sensitive institutional information. The human evaluation (Experimental Setup, Results) used two volunteer bilingual annotators who rated model outputs. Each annotator was informed of the purpose of the rating task, that their participation was voluntary and unpaid, and that their ratings would be reported only in aggregate, before they began; this constitutes informally obtained informed consent, though no separate signed consent form was administered. No personal, demographic, or identifying data about the annotators was collected or is reported. We believe this places the task within the kind of standard, low-risk NLP annotation activity many institutions exempt from formal ethics-committee review, but this has not been independently confirmed with the author's institutional ethics office, and we state that plainly rather than asserting approval was not required; seeking a formal exemption or determination remains an open action item ahead of publication.

AI usage disclosure

Large language model assistance (Anthropic Claude) was used for software implementation (the LSPM middleware, evaluation scripts, statistical analysis scripts, ARCD sampling and Arabic EM/F1 scoring code), for literature-search assistance in identifying candidate related work subsequently verified by the author against primary sources, and for drafting assistance on this manuscript under the author's direction and review. All experimental results reported in this paper were produced by executing the released code against live infrastructure; no results were generated, adjusted, or selectively reported without corresponding code execution, and non-significant and inconclusive findings are reported in full rather than omitted. The author takes full responsibility for the accuracy of all claims, citations, and reported results in this paper.

Data availability

The available datasets, evaluation outputs, aggregated GPU benchmark results, and implementation are publicly released at https://github.com/Akramtaha98/vllm-arabic-rag: the LSPM middleware, evaluation scripts, statistical analysis scripts, and benchmarking harness; the tokenization and fidelity/baseline pilot datasets; the ARCD ground-truth re-test dataset and code; the end-to-end retrieval check dataset and code; and, for the GPU benchmark, the aggregated 35-cell summary and comparison figure. The underlying per-request logs from the preliminary GPU run were generated on a temporary rented cloud instance and were not preserved before this submission; this is stated plainly rather than implied complete. An earlier commit of this repository was tagged v1.0-submission and archived on Zenodo (DOI: 10.5281/zenodo.21826992); that snapshot predates this submission's changes, and the version-specific DOI cited here should be understood as covering that earlier commit, not the manuscript as currently submitted, until a new tagged release and Zenodo version are created.

References

[1] A. Vaswani et al., "Attention Is All You Need", Proc. of Advances in Neural Information Processing Systems, 2017, doi: 10.48550/arXiv.1706.03762.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", Proc. of Advances in Neural Information Processing Systems, 2020, doi: 10.48550/arXiv.2005.11401.

[3] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention", Proc. of the 29th ACM Symp. on Operating Systems Principles, 2023, doi: 10.1145/3600006.3613165.

[4] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks", Proc. of Int. Conf. on Learning Representations, 2024, doi: 10.48550/arXiv.2309.17453.

[5] Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models", Proc. of Advances in Neural Information Processing Systems, 2023, doi: 10.52202/075280-1506.

[6] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models", Proc. of Conf. on Empirical Methods in Natural Language Processing, 2023, doi: 10.18653/v1/2023.emnlp-main.825.

[7] Y. Li, B. Dong, C. Lin, and F. Guerin, "Compressing Context to Enhance Inference Efficiency of Large Language Models", Proc. of Conf. on Empirical Methods in Natural Language Processing, 2023, doi: 10.18653/v1/2023.emnlp-main.391.

[8] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation", Proc. of Int. Conf. on Learning Representations, 2024, doi: 10.48550/arXiv.2310.04408.

[9] R. Nogueira and K. Cho, "Passage Re-ranking with BERT", arXiv preprint arXiv:1901.04085, 2019, doi: 10.48550/arXiv.1901.04085.

[10] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering", Proc. of Conf. on Empirical Methods in Natural Language Processing, 2020, doi: 10.18653/v1/2020.emnlp-main.550.

[11] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT", Proc. of the 43rd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, 2020, doi: 10.1145/3397271.3401075.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", Proc. of Conf. on Empirical Methods in Natural Language Processing, 2019, doi: 10.18653/v1/D19-1410.

[13] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding", Proc. of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, 2020, doi: 10.48550/arXiv.2003.00104.

[14] H. Mozannar, E. Hajal, E. Maamary, and H. Hajj, "Neural Arabic Question Answering", Proc. of the Fourth Arabic Natural Language Processing Workshop, 2019, doi: 10.18653/v1/W19-4612.

[15] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models", Proc. of the 16th USENIX Symp. on Operating Systems Design and Implementation, 2022. (No DOI on primary source.)

[16] V. Setiawan and B. Soewito, "Instruction-based Chain-of-Thought for Multi-agent RAG in Snort Rule Generation", International Journal of Intelligent Engineering and Systems, vol. 18, no. 11, pp. 883-894, 2025, doi: 10.22266/ijies2025.1231.54.

[17] S. Vaidya, J. R. Saini, and P. A. Tamhankar, "A Novel Approach for Enhancing Large Language Model Reliability through Evolutionary Hallucination Risk Modeling with H-Tree Phylogeny", International Journal of Intelligent Engineering and Systems, vol. 18, no. 11, pp. 863-882, 2025, doi: 10.22266/ijies2025.1231.53.

[18] V. Kataria, "Intelligent Site Reliability Engineering: A Multi-agent LLM Framework for Automated Incident Analysis and Root Cause Determination", International Journal of Intelligent Engineering and Systems, vol. 18, no. 11, pp. 450-466, 2025, doi: 10.22266/ijies2025.1231.28.